

Final Exam Solutions

Statistical Methods, Lab Exam(통계적방법론, 실기시험)

2026 1st semester

Year(학년)	4	Cadet number(교번)		Name		Score	
----------	---	------------------	--	------	--	-------	--

<Instructions>

1. It is open-book, closed-web test.
2. You are not allowed to discuss on the problems during the exam.
3. Import *housing.csv* data, converting all the string variables to factors, and import *fci_labels_2_3.csv* data without converting the string variables to factors.

Problem 1 [40 points]

We aim to predict the continuous variable **price** in *housing.csv* using the remaining variables with a random forest model.

- (1) Import *housing.csv* dataset, converting all the string variables to factors. Use the **str** function to examine the structure of the *housing* data. How many observations, how many variables, and how many factor variables does the dataset contain?

<R-code>

```
> str(housing)
```

<Output>

```
'data.frame':    545 obs. of  13 variables:
 $ price          : int   13300000 12250000 12250000 12215000 11410000 10850000 10150000
10150000 9870000 9800000 ...
 $ area           : int   7420 8960 9960 7500 7420 7500 8580 16200 8100 5750 ...
 $ bedrooms       : int    4 4 3 4 4 3 4 5 4 3 ...
 $ bathrooms      : int    2 4 2 2 1 3 3 3 1 2 ...
 $ stories        : int    3 4 2 2 2 1 4 2 2 4 ...
 $ mainroad       : Factor w/ 2 levels "no","yes": 2 2 2 2 2 2 2 2 2 2 ...
 $ guestroom      : Factor w/ 2 levels "no","yes": 1 1 1 1 2 1 1 1 2 2 ...
 $ basement       : Factor w/ 2 levels "no","yes": 1 1 2 2 2 2 1 1 2 1 ...
 $ hotwaterheating : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 1 1 1 ...
 $ airconditioning : Factor w/ 2 levels "no","yes": 2 2 1 2 2 2 2 1 2 2 ...
 $ parking        : int    2 3 2 3 2 2 2 0 2 1 ...
 $ prefarea       : Factor w/ 2 levels "no","yes": 2 1 2 2 1 2 2 1 2 2 ...
 $ furnishingstatus: Factor w/ 3 levels "furnished","semi-furnished",...: 1 1 2 1 1 2 2 3 1 3 ...
```

The number of observations: 545

The number of variables: 13

The number of factor variables: 7

- (2) Randomly select **300** observations from the *housing* data and save them as the training data(**housing.train**). Save the remaining observations as the test data(**housing.test**).

<R-code>

```
n = nrow(housing)
Index = sample(1:n, 300)
housing.train = housing[Index,]
housing.test = housing[-Index,]
```

- (3) Using the training data, fit a **random forest** model with **price** as the response variable and all other variables as explanatory variables. (Set the number of variables randomly selected at each split to **4**.) Using the test data, obtain the predicted response variable. (A) How many trees were used in total in the random forest? (B) Using **head** function, find the **observed** and **predicted** price for the **first** object in **test** data.

<R-code>

```
library(randomForest)
model2 = randomForest(price ~ ., data = housing.train,
                      mtry = 4)
pred = predict(model2, housing.test)
true = housing.test$price
head(true)
head(pred)
```

<Output>

```
> model2
Call:
randomForest(formula = price ~ ., data = housing.train, mtry = 4)
Type of random forest: regression
Number of trees: 500

> head(true)
[1] 12250000 10850000 10150000 9310000 9100000 8960000

> head(pred)
      3      6      7     13     16     18
7278749 7805816 9374455 7830290 5593111 7890101
```

Number of trees: 500

Observed price for the first object in test data: 12250000

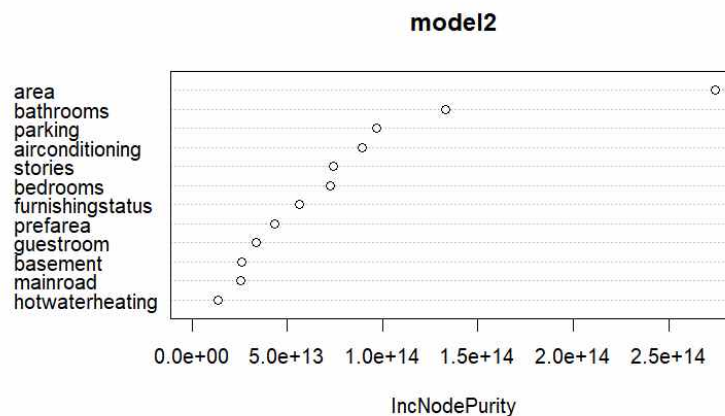
Predicted price for the first object in test data: 7278749

- (4) For the random forest model, visualize the **variable importance**. Which variable is the most important for predicting **price**?

<R-code>

```
varImpPlot(model2)
```

<Output (plot)>



The most important variable: area

Problem 2 [40 points]

We try to predict the binary variable with two levels *[question]* and *[command]* in *fci_labels_2_3.csv* using Korean sentences with a neural network model. Import *fci_labels_2_3.csv* data without converting the string variables to factors.

<Baseline Code> - Run This First (Do not change the R code)

```
library(keras3)
set.seed(41)
review = fci_labels_2_3
colnames(review) <- c("score", "comment")
review$score <- as.integer(review$score)
review <- na.omit(review) # Remove NAs
review$score = ifelse(review$score == 2, 0, 1) # Recode labels 2/3 -> 0/1
# -----
# Train / valid / test split
# -----
n <- nrow(review) %/% 10
idx <- sample.int(nrow(review), n) # sample n rows from the WHOLE dataset (not just 1:n)
review <- review[idx, ]
n_test <- max(1, round(0.20 * n))
test_set <- review[seq_len(n_test), ]
train_set <- review[-seq_len(n_test), ]
x_train <- train_set$comment
x_test <- test_set$comment
y_train <- train_set$score
y_test <- test_set$score
# -----
# Text vectorization (minimal preprocessing)
# -----
# - Keeps text as-is (no lowercasing/stripping) because it's Korean.
max_words <- 20000L
maxlen <- 50L # you can raise to reduce truncation
vec <- layer_text_vectorization(
  max_tokens = max_words,
  output_mode = "int",
  output_sequence_length = maxlen,
  standardize = NULL # keep Korean punctuation/spacing as-is
)# Build the vocabulary from training text only
adapt(vec, train_set$comment)
# Save the whole vocabulary (not necessary)
vocab <- keras3::get_vocabulary(vec); head(vocab, 30)
# -----
# Model: TextVectorization -> Embedding -> CNN
# -----
embed_dim <- 128L
model <- keras3::keras_model_sequential(
  layers = list(
    vec,
    keras3::layer_embedding(input_dim = max_words, output_dim = embed_dim, mask_zero = FALSE),
    keras3::layer_spatial_dropout_1d(rate = 0.2),
    keras3::layer_conv_1d(filters = 128, kernel_size = 5, activation = "relu", padding = "same"),
    keras3::layer_conv_1d(filters = 128, kernel_size = 5, activation = "relu", padding = "same"),
    keras3::layer_global_max_pooling_1d(),
    keras3::layer_dropout(rate = 0.4),
    keras3::layer_dense(units = 64, activation = "relu"),
    keras3::layer_dropout(rate = 0.3),
    keras3::layer_dense(units = 1, activation = "sigmoid")
  )
)
model |> compile(
  optimizer = "adam",
  loss = "binary_crossentropy",
  metrics = "accuracy"
)# -----
# Train
# -----
history <- model |>
fit(
  x = x_train, y = y_train,
  epochs = 5,
  batch_size = 32,
  validation_split = 0.2,
  verbose = 2
)
```

- (1) Print a readable model summary. Based on the baseline code and R output, fill in the blanks with the most appropriate words or values.

<R-code>

```
> summary(model)
```

<Output>

Model: "sequential_2"

Layer (type)	Output Shape	Param #
text_vectorization_2 (TextVectorization)	(None, 50)	0
embedding_2 (Embedding)	(None, 50, 128)	2,560,000
spatial_dropout1d_2 (SpatialDropout1D)	(None, 50, 128)	0
conv1d_4 (Conv1D)	(None, 50, 128)	82,048
conv1d_5 (Conv1D)	(None, 50, 128)	82,048
global_max_pooling1d_2 (GlobalMaxPooling1D)	(None, 128)	0
dropout_4 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 64)	8,256
dropout_5 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 1)	65
Total params: 8,197,253 (31.27 MB)		
Trainable params: 2,732,417 (10.42 MB)		
Non-trainable params: 0 (0.00 B)		
Optimizer params: 5,464,836 (20.85 MB)		

The response variable y is (continuous / [categorical])

The number of total parameters: 8,197,253

The activation function was used in the output layer: ([sigmoid] / ReLU / linear)

Dropout (increases / [decreases]) the number of total parameters.

Input dimension of the embedding layer: 20000

Output dimension of the embedding layer: 128

Optimizer: adam

Number of epochs = 5

Number of samples per gradient update (batch size) = 32

How do you get 8,256 parameters in the last hidden layer? $8,256 = \underline{128 * 64 + 128}$

- (2) Compute predicted probabilities, convert them to class labels using threshold **0.5**. Then, create a **confusion matrix**. Also, fill in the blanks with the most appropriate words.

```
<R-code>
pred_prob <- model |> predict(x_test)
pred_lab <- as.integer(pred_prob > 0.5)
print(table(Predicted = pred_lab, Actual = y_test))
mean(pred_lab== y_test)
sum(y_test == 1 & pred_lab == 1) / sum(y_test == 1)
```

```
<Output>
      Actual
Predicted  0   1
      0 336 100
      1  92 452
> mean(pred_lab== y_test)
[1] 0.804
> sum(y_test == 1 & pred_lab == 1) / sum(y_test == 1)
[1] 0.819
```

Accuracy: 0.804

Sensitivity: 0.819

As the threshold increases, sensitivity (increases / [decreases]) and specificity ([increases] / decreases).

After training, the neural network achieves (global minimum / [local minimum]).

- (3) Print the labels, predicted probabilities, and raw text for the **two** observations predicted to be the most likely be classified as class 0. Based on the output, what does class 0 represent: question or command?

```
<R-code>
eg_idx <- head(order(pred_prob), 2)
for (i in eg_idx) {
  cat("\n--- Example -----Wn")
  cat("True:", y_test[i], "Wn")
  p <- as.numeric(model |> predict(x_test[i], verbose = 0))
  cat(sprintf("Pred prob (positive=1): %.3fWn", p))
  cat("Text:", x_test[i], "Wn")
}
```

```
<Output>
--- Example -----
True: 0
Pred prob (positive=1): 0.409
Text: 이번 주의 초미세먼지 수치를 알려줘

--- Example -----
True: 0
Pred prob (positive=1): 0.410
Text: 부재중 전화가 몇 통 왔는지 알려줘
```

Class 0: ([Question] / Command)

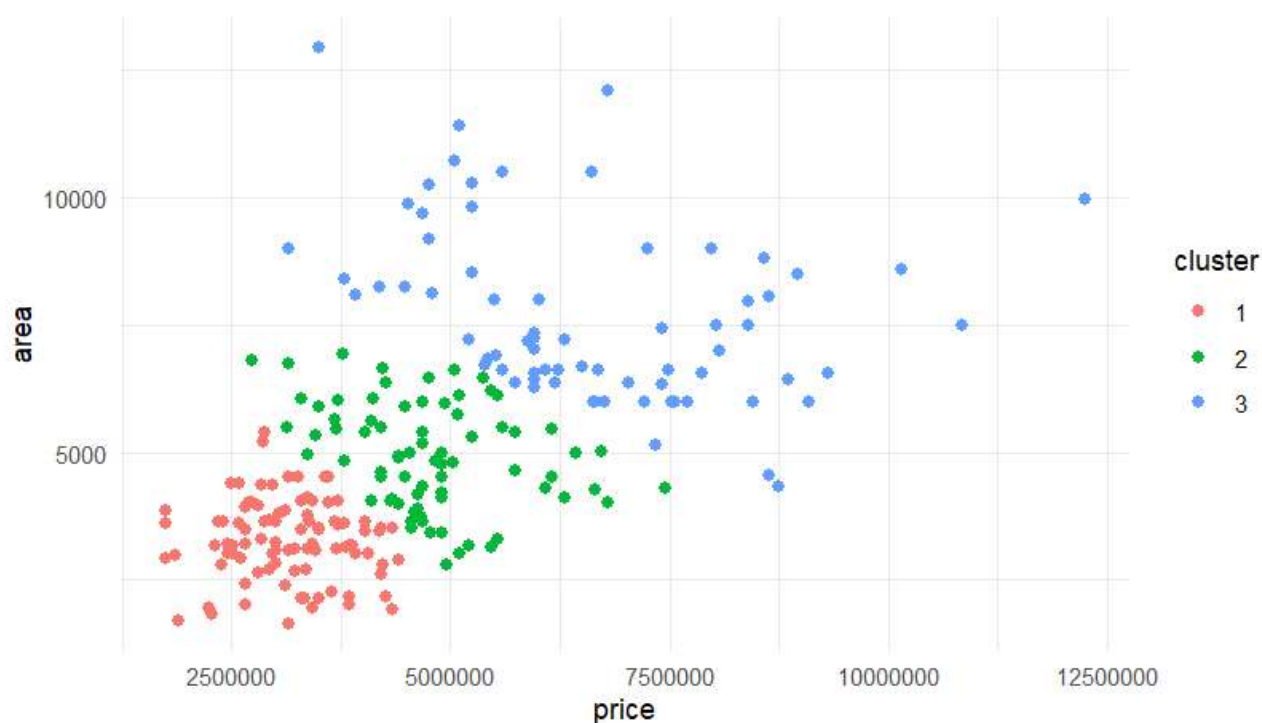
Problem 3 [20 points]

Using the `housing.test` dataset in Problem 1, perform K-Means clustering with $k = 3$ on the `price` and `area` variables. Before clustering, standardize both variables using standard scaling. Then, using the `ggplot2` package, create a scatter plot with `price` on the x-axis and `area` on the y-axis, coloring the points by cluster (three distinct colors). Using the scatter plot, identify which cluster has the highest average price and which cluster has the largest variance of price. Also, fill in the blanks with the most appropriate words.

<R-code>

```
df = housing.test[c("price", "area")]
km = kmeans(scale(df), centers = 3)
library(ggplot2)
df$cluster <- as.factor(km$cluster)
ggplot(df, aes(x = price, y = area, color = cluster)) +
  geom_point(size = 2) +
  theme_minimal()
```

<Output (plot)>



The highest average price: (Cluster 1 / Cluster 2 / [Cluster 3])

The largest variance: (Cluster 1 / Cluster 2 / [Cluster 3])

K-means tries to (maximize / [minimize]) the total ([within-cluster] / between-cluster) variation.